

DAY 14

1. Command Chaining with List Operators

Command chaining means executing multiple commands in a controlled sequence using Bash **list operators**.

The important operators are:

Operator	Name	Meaning
&	Background execution	Run command in background
&&	Logical AND	Run next command only if previous succeeds
	Logical OR	The OR operator means the second command only runs if the first one failed
;	Sequential execution	Run commands regardless of previous result

& – Background Execution

The `&` operator tells Bash to run a command **in the background**.

```
sleep 30 &
```

Instead of waiting 30 seconds, the shell immediately gives you the prompt back.

You can continue using the terminal:

```
sleep 30 &  
echo "I can continue working"
```

Mental model

```
Command → Background  
↓
```

Shell remains usable

Important: `&` does not mean "run the next command only if..." It simply starts the preceding command asynchronously.

`&&` – Logical AND

```
command1 && command2
```

`command2` executes **only when** `command1` **succeeds**.

In Bash, success normally means **exit status** `0`.

Example:

```
mkdir backup && echo "Backup created"
```

If `mkdir backup` succeeds:

```
mkdir backup → success (0)
                ↓
            echo executes
```

If it fails:

```
mkdir backup → failure
                ↓
            echo does NOT execute
```

Common use

```
command && echo "Success"
```

This is very useful in automation.

`||` – Logical OR

```
command1 || command2
```

`command2` executes **only if** `command1` **fails**.

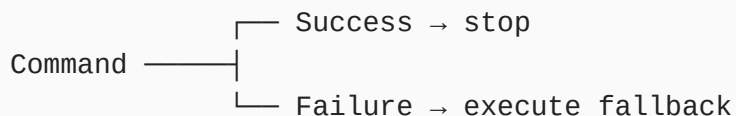
Example:

```
mkdir backup || echo "Could not create backup"
```

If `mkdir` succeeds, the `echo` doesn't run.

If `mkdir` fails, the `echo` runs.

Mental model



A common pattern is:

```
command && echo "Success" || echo "Failed"
```

This can be useful, but remember that the middle command's own exit status can affect the `||` behavior. For complicated logic, an `if` statement is usually clearer.

; – Sequential Execution

```
command1 ; command2
```

The second command executes **regardless of whether the first command succeeds or fails**.

Example:

```
echo "Start" ; false ; echo "Still runs"
```

Output will include:

```
Start  
Still runs
```

Even though `false` returns a failure status, the next command still executes.

Mental model

```
Command 1  
  ↓  
Command 2  
  ↓  
Command 3
```

No success/failure condition is imposed.

2. The `test` Command

The `test` command is one of the foundations of **Bash conditional logic**. It evaluates a condition and returns an **exit status**.

```
test condition
```

You will frequently see it written as:

```
[ condition ]
```

These are essentially two syntactic forms for testing conditions.

Very important

There must be spaces:

```
[ "$x" -eq 10 ]
```

Correct.

This is wrong:

```
["$x" -eq 10]
```

Because `[` is treated as a command name and requires the condition to be separated from it.

Exit Status

Bash commands return an **exit status** when they finish.

Exit status	Meaning
0	Success / condition true
Non-zero	Failure / condition false

For the `test` command:

```
[ 10 -eq 10 ]
```

returns:

```
0
```

while:

```
[ 10 -eq 20 ]
```

returns a non-zero status.

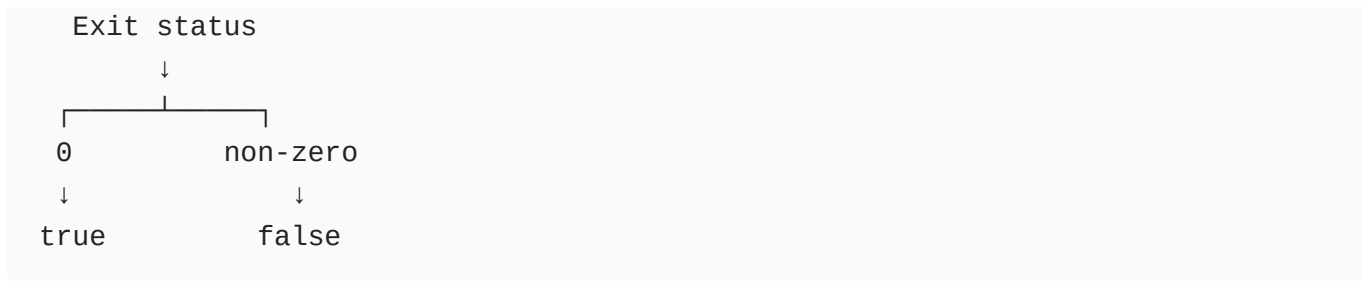
You can inspect it using:

```
echo $?
```

`$?` contains the exit status of the **most recently executed command**.

This connects everything together

```
[test condition]  
↓
```



This is why `&&`, `||`, and `if` statements work.

3. Integer Comparisons

When comparing **numbers**, Bash uses special integer comparison operators.

Operator	Meaning	Example
<code>-eq</code>	Equal	<code>5 -eq 5</code>
<code>-ne</code>	Not equal	<code>5 -ne 3</code>
<code>-gt</code>	Greater than	<code>5 -gt 3</code>
<code>-lt</code>	Less than	<code>3 -lt 5</code>
<code>-ge</code>	Greater than or equal	<code>5 -ge 5</code>
<code>-le</code>	Less than or equal	<code>5 -le 10</code>

Example:

```
a=10
b=20

[ "$a" -lt "$b" ]
```

The condition is true because:

```
10 < 20
```

Important distinction

Don't confuse:

```
-eq
```

with:

```
==
```

`-eq` is primarily for **integer comparison**.

4. String Testing

Strings are text values.

Example:

```
name="Sourya"
```

You can compare strings using:

```
[ "$name" == "Sourya" ]
```

or:

```
[ "$name" != "Rahul" ]
```

Important string operators

Operator	Meaning
<code>==</code>	Strings are equal
<code>!=</code>	Strings are different
<code>-z</code>	String is empty
<code>-n</code>	String is not empty

Example

```
name=""
```

```
[ -z "$name" ]
```

True because `name` is empty.

Another:

```
name="Sourya"  
[ -n "$name" ]
```

True because the string is not empty.

Best practice

Quote variables in tests:

```
[ "$name" == "Sourya" ]
```

rather than relying on:

```
[ $name == Sourya ]
```

Quoting protects against spaces and empty values causing unexpected parsing.

5. File Testing

Bash can test properties of files and directories.

This is extremely useful in scripts.

Operator	Checks
<code>-e</code>	Path exists
<code>-f</code>	Regular file
<code>-d</code>	Directory
<code>-x</code>	Executable permission
<code>-r</code>	Readable
<code>-w</code>	Writable

-e – Exists

```
[ -e "$file" ]
```

Checks whether the specified path exists.

-f – Regular File

```
[ -f "$file" ]
```

Checks whether the path is a **regular file**.

For example:

```
report.txt → regular file
```

-d – Directory

```
[ -d "$directory" ]
```

Checks whether the path is a directory.

-x – Executable

```
[ -x "$file" ]
```

Checks whether the current context has execute permission for the file.

-r – Readable

```
[ -r "$file" ]
```

Checks whether the file is readable.

-w – Writable

```
[ -w "$file" ]
```

Checks whether the file is writable.
